網路應用軟體設計期末專題：多人幾A幾B網路對戰系統

Oct 8, 2026 · @黃崇源（Chung-Yuan Huang）

本專題要求每組三人以 Python 3 實作一支可同時扮演伺服器（莊家）、人類玩家客戶端與 AI 代理人的程式，並嚴格遵守本文件定義的 BC/1.0 應用層通訊協定，使任何兩組的程式都能跨越網際網路直接連線對玩。本規範於 Oct 20, 2026 公告，各組於 Dec 22, 2026 繳交成果並進行全班跨組對玩。

第一部分　作業規範

1. 專題目標與學習成果

本專題的核心要求是「互通性」：全班各組實作的程式，只要遵守同一份 BC/1.0 協定，就必須能夠彼此連線、共同完成一局多人猜數字遊戲。題材取自經典的「公牛與母牛」（Bulls and Cows）遊戲。這個紙筆遊戲早於商業桌遊 Mastermind，後者即以它為藍本；最早的電腦版本則是 1970 年前後劍橋大學在 Titan 主機上執行的 MOO（Wikipedia contributors, n.d.）。

本專題把傳統的兩人對猜，擴充為 2 至 6 人同場競技。輪到的玩家喊出一個四位數，其餘每位玩家都要依自己的秘密數字回報幾 A 幾 B，回報結果則公開給全場。因此，每一次喊數都會同時洩漏所有回報者的資訊。

完成本專題後，同學應能掌握以下能力：以 TCP 通訊端（socket）設計並實作應用層協定；以多執行緒或 asyncio 處理多個連線與使用者輸入的並行；在不可信任的對手面前維持程式的穩健性；運用密碼學雜湊的承諾機制，在沒有可信第三方的情況下確保遊戲公平；以候選集合刪減與搜尋策略設計 AI 代理人。

本文件是全班唯一的共同標準。若本文件與課程提供的參考伺服器行為不一致，請立即回報助教。課程將以勘誤公告修正文件，並同步更新版本紀錄；各組不得自行擴充或改寫協定。

2. 遊戲規則

2.1 秘密數字與喊數。 秘密數字與喊出的數字都必須是四個 ASCII 數字字元，四個位數兩兩相異，允許首位為 0，例如 `0275`。合法數字共有 10 × 9 × 8 × 7 = 5040 個。在這個設定下，已知任何秘密數字都能在 7 次內猜中（Wikipedia contributors, n.d.），而期望猜測次數最少的策略平均約需 5.213 次（Tanaka, n.d.）。不同玩家的秘密數字可以相同；一旦相同，一次喊數可能同時猜中多人。

2.2 幾 A 幾 B 的計算。 設喊數 g 與秘密數字 s 的第 i 位數字分別為 g[i] 與 s[i]（i = 1, 2, 3, 4）。A 是「數字與位置皆正確」的個數，B 是「數字正確但位置不對」的個數：

```
A = | { i ∈ {1,2,3,4} : g[i] = s[i] } |
B = | {g[1], g[2], g[3], g[4]} ∩ {s[1], s[2], s[3], s[4]} | − A
```

例如秘密數字為 `0275`、喊數為 `0927` 時，0 的位置正確，2 與 7 存在但位置不對，結果為 1A2B。合法的回報必須滿足 A + B ≤ 4，而且不可能出現 3A1B：三個位置已對時，第四個數字若也在秘密數字中，只能落在剩下的位置而成為 4A。因此合法回報恰有 14 種。

2.3 回合流程。 一局遊戲依下列步驟進行：

1. 伺服器於開局時隨機決定輪轉順序並公告，每位玩家設定秘密數字並送出承諾值（第 3 節）。
2. 依順序輪到下一位存活玩家，由他喊出一個合法四位數。
3. 其餘每位存活玩家依自己的秘密數字回報幾 A 幾 B；喊數者不回報自己的喊數。
4. 伺服器收齊所有回報（或逾時）後，一次將全部回報公開給所有人，包括 AI 代理人與已淘汰但仍在線上的玩家。
5. 回報 4A0B 的玩家「被猜中」而淘汰，此後不再喊數、不再回報，但可繼續觀看。
6. 無論是否猜中任何人，都輪到下一位存活玩家，重複步驟 2 至 5。
7. 存活玩家只剩一位或已無人存活，或回合數達到上限 `max_turns` 時，進入終局。

由於回報全部公開，每位玩家都可以利用他人喊數所得的資訊。一個好的喊數應同時縮小多位對手的候選範圍，這正是多人版本比兩人版本更有策略深度之處。

2.4 名次。 最早被猜出者名次最後，最後留在場上者獲勝。為處理斷線、逾時與違規，名次依下表的終局狀態決定，並採用標準競賽排名（並列者同名次，下一名次跳號，例如 1、2、2、4）。

終局狀態	意義	名次規則
`SURVIVOR`	終局時仍存活，且揭示驗證通過	第 1 名；因回合上限而結束時，所有存活者並列第 1 名
`GUESSED`	被猜中淘汰，且揭示驗證通過	排在存活者之後；淘汰回合越晚名次越前，同回合並列
`FORFEIT`	斷線、回報逾時，或連續未喊數達上限	排在所有 `GUESSED` 之後；棄權回合越晚名次越前
`DQ`	回報自相矛盾、揭示與承諾不符，或未揭示	取消名次

例如四人對局中，甲存活到最後，乙與丙同在第 9 回合被猜中，丁在第 12 回合斷線；名次依序為甲 1、乙 2、丙 2、丁 4。丁雖然較晚離場，但棄權者的回報無法經揭示驗證，故排在被猜中者之後。這個規則也讓「故意斷線以逃避驗證」無利可圖。

3. 公平機制：承諾—揭示與公開一致性檢查

幾 A 幾 B 由玩家自行回報，伺服器並不知道任何人的秘密數字，因此無法當場判斷回報是否誠實。若不加防範，玩家可能輸入錯誤，甚至永遠不承認 4A，使遊戲無法公正收斂。本協定不要求玩家把秘密交給伺服器，而是採用密碼學的承諾—揭示（commit–reveal）機制（Blum, 1983）。如此一來，別組撰寫的莊家程式不必被信任，也看不到任何人的秘密。

3.1 承諾值。 開局時，每位玩家以密碼學安全的亂數產生器產生 16 位元組的鹽值 r（以 32 個小寫十六進位字元表示），並只送出以下的 SHA-256 承諾值（National Institute of Standards and Technology, 2015）：

```
c = SHA256( s + ":" + r )        # + 為字串串接
```

其中 s 為四位秘密數字，r 為鹽值，整個字串以 ASCII 編碼後計算雜湊，結果以 64 個小寫十六進位字元傳送。Python 的寫法如下，鹽值必須用 `secrets` 模組產生，不可用 `random`（Python Software Foundation, n.d.-c）：

```
import hashlib, secrets
salt = secrets.token_hex(16)                        # 32 個小寫十六進位字元
commit = hashlib.sha256(f"{secret}:{salt}".encode("ascii")).hexdigest()
```

各組請以下列測試向量驗證自己的實作：秘密數字 `0275`、鹽值 `2eb6594c74b9b9d71e2d9b5bd14300df`，承諾值必須是 `6b05c8fe4113aa24833fcbd089d33f0fe48ff68eb0a7d7fff87319a394338721`。

鹽值不可省略。合法秘密數字只有 5040 個，若只對秘密數字本身取雜湊，對手只要把 5040 個雜湊值算一遍就能反查出秘密。加入 128 位元的隨機鹽值後，窮舉變得不可行，這稱為承諾的隱藏性。另一方面，SHA-256 的抗碰撞性使玩家無法在事後找到另一組秘密與鹽值來符合同一個承諾值，這稱為綁定性。

3.2 三道防線。 協定以下列三道檢查，讓不實回報必然被發現：

1. 公開一致性檢查（每回合）。對每位回報者，伺服器必須計算 5040 個合法數字中，仍與該玩家所有歷史回報相容者。若候選集合變成空集，證明他至少說謊一次，當回合立即取消資格。由於所有回報都公開，任何客戶端也能自行重算。
2. 淘汰時揭示。玩家回報 4A0B 而被判定猜中後，伺服器會送出揭示要求，程式收到後必須立即送出秘密數字與鹽值；伺服器驗證承諾值，並以揭示的秘密重算他過去的每一筆回報。
3. 終局揭示。終局時仍存活的玩家同樣揭示並接受驗證。任何一項驗證失敗者一律取消名次，其餘玩家名次遞補。

此外，伺服器在收齊本回合所有回報之前，不得洩漏任何人的回報，以免玩家看了別人的答案再決定如何回報。客戶端也應自行驗證伺服器廣播的每一筆揭示結果，而不是盲目接受判定。人類玩家客戶端應以儲存的秘密數字自動算出建議回報，由玩家確認後送出；玩家仍可修改數值，但不實回報將在上述驗證中被取消資格。

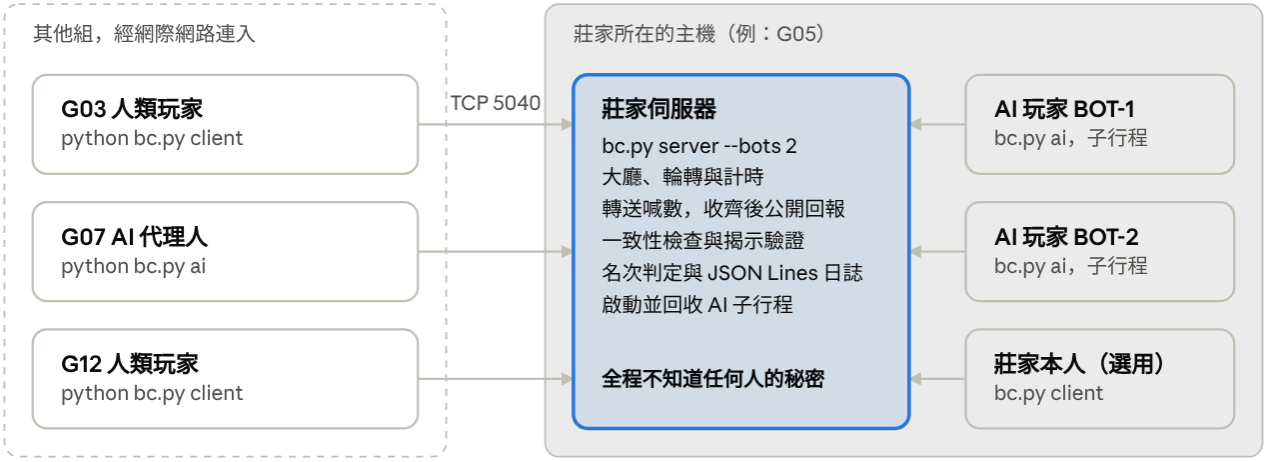
4. 系統架構與三種身分

系統採用星狀的主從式（client-server）架構。莊家就是伺服器，所有玩家（包括 AI 代理人）都以客戶端身分經由 TCP 連入。伺服器只擔任裁判：管理大廳與輪轉順序、轉送喊數、收齊並公開回報、執行一致性檢查、計時與判定名次。它從頭到尾不知道任何人的秘密數字。

每組只繳交一支程式，以子命令決定身分。莊家以 `--bots K` 指定 AI 玩家時，伺服器自動以子行程啟動 K 個本組程式的 `ai` 身分，連到本機 127.0.0.1。如此，AI 玩家與遠端玩家走完全相同的協定，任一組的 AI 也能加入任一組的伺服器。莊家本人若要下場，只需在另一個終端機視窗執行自己的 `client`。

身分	子命令	主要職責
伺服器（莊家）	`server`	大廳與輪轉、轉送喊數、收齊並公開回報、一致性檢查、逾時與名次、日誌、啟動 AI 子行程
人類玩家客戶端	`client`	設定秘密、喊數、確認回報、顯示所有公開資訊；自動送出承諾與揭示
AI 代理人	`ai`	自動完成玩家的所有動作，依公開資訊推論各對手的秘密數字

所有玩家（包括 AI）都以同一個 BC/1.0 協定連到莊家伺服器



BC/1.0 系統架構 · 一個莊家伺服器與六個玩家連線

所有箭頭都由客戶端主動連向伺服器。AI 子行程與遠端玩家走的是同一個協定，差別只在於連線來自本機。

5. 程式功能要求

5.1 執行環境。 程式以 Python 3.10 以上版本撰寫，必須能在 Windows、macOS 與 Linux 執行。除標準函式庫外也可使用第三方套件，但必須列在 `requirements.txt` 並標明版本，且不得改變線上的訊息格式。程式進入點一律命名為儲存庫根目錄的 `bc.py`。介面可以是圖形介面，也可以是文字主控台。

5.2 統一命令列。 為了讓助教以自動化工具測試每一組，下列子命令與選項為必要功能，名稱與預設值不得更改：

```
python bc.py server [--host 0.0.0.0] [--port 5040] [--players N] [--bots K]
                    [--guess-timeout 60] [--feedback-timeout 60] [--commit-
timeout 30]
                    [--reveal-timeout 15] [--max-turns 120] [--max-skips 3]
                    [--games 1] [--log-dir logs]
python bc.py client --host HOST [--port 5040] --name NAME [--team G00]
python bc.py ai     --host HOST [--port 5040] --name NAME [--team G00] [--
quiet]
```

選項	預設值	意義
`--host`（server）	`0.0.0.0`	伺服器監聽的位址
`--port`	`5040`	TCP 埠號
`--players`	`2`	本局玩家總數（2–6，含 AI）；人數到齊即自動開局
`--bots`	`0`	伺服器自動啟動的 AI 玩家數，0 ⩽ K ⩽ N
`--guess-timeout`	`60`	喊數時限（秒）
`--feedback-timeout`	`60`	回報時限（秒）
`--commit-timeout`	`30`	送出承諾值時限（秒）
`--reveal-timeout`	`15`	揭示時限（秒）
`--max-turns`	`120`	回合數上限
`--max-skips`	`3`	連續逾時未喊數幾次即視為棄權
`--games`	`1`	連續舉辦的局數，0 表示不限
`--log-dir`	`logs`	日誌目錄
`--host`（client、ai）	必填	伺服器位址
`--name`	必填	玩家名稱，1–20 字，同局不可重複
`--team`	`G00`	組別代碼，例如 `G07`

各組可以增加其他選項，例如圖形介面開關，但不加這些選項時的行為必須如上表。ai 另有 --quiet 選項可減少輸出；所有時限選項都必須接受小數秒數，例如 1.5。伺服器應另提供主控台命令 `start`，讓莊家在至少 2 人加入時提早開局。

5.3 伺服器。 伺服器必須做到以下各項：

1. 同時服務多個連線；單一連線的錯誤、緩慢或惡意行為不得拖累其他玩家，也不得使伺服器當機。
2. 完整實作第二部分的狀態機、訊息、逾時、錯誤碼與名次規則。
3. 每回合執行公開一致性檢查，且在收齊回報前不洩漏任何回報。
4. 依 `--bots` 啟動 K 個本組程式的 `ai` 子行程，名稱依序為 `BOT-1` 至 `BOT-K`；`--host` 為 `0.0.0.0` 時連到 `127.0.0.1`，否則連到 `--host` 指定的位址。一局結束後回收這些子行程。
5. 每局寫出一個日誌檔（5.6 節）。
6. 一局結束後關閉該局所有連線；`--games` 大於 1 時重新開放大廳，主持完指定局數後行程必須自行結束。

7. 監聽通訊端應設定 `SO_REUSEADDR`，使伺服器結束後能立即在同一埠號重新啟動。
8. 只有在標準輸入是終端機時才讀取主控台命令；標準輸入為空或已關閉時仍必須正常運作。

5.4 人類玩家客戶端。 客戶端必須做到以下各項：

1. 網路接收與使用者輸入必須並行：等待輸入時仍要即時顯示新訊息，並自動回應 `PING`。
2. 開局時讓玩家輸入秘密數字或隨機產生，程式自動計算並送出承諾值；鹽值只存在記憶體中。
3. 輪到喊數時提示玩家輸入，先在本地檢查格式，並應顯示剩餘時間。
4. 需要回報時，顯示對方的喊數與自己的秘密，預填自動計算的結果，由玩家確認後送出；玩家修改數值時必須再次確認。
5. 收到 `REVEAL_REQUEST` 時由程式自動揭示，不需人類介入。
6. 清楚顯示大廳、輪轉順序、每回合的全部回報、淘汰、揭示判定與最終名次；建議一併提供各對手的剩餘候選數與聊天功能。

5.5 AI 代理人。 AI 必須全自動運作，並在時限內完成每個動作，建議每次喊數在 2 秒內決定。AI 應為每位存活對手維護一個候選集合，並以每一則 `TURN_RESULT` 更新，而不只是自己喊數的結果。基準策略是鎖定候選最少的對手，從其候選中挑選能把候選切成最多組的數字，也就是 Kooi (2005) 的「最多劃分」（most parts）準則。

各組可以往更強的方向發展，例如 Knuth (1976) 的極小化最壞情況（minimax）、最小化期望剩餘候選數，或最大化資訊熵。多人局的特殊之處在於：一次喊數同時測試所有對手，而且只有喊中某人的候選才有機會當場猜中他。因此 AI 必須在「累積對全場的資訊」與「立即猜中一人」之間取捨。更進一步的策略還可考慮輪轉順序：下一位喊數者也會利用你創造的資訊。各組需在報告中以至少 100 局實驗，呈現 AI 對參考 AI 的勝率、平均名次與平均回合數。

5.6 日誌。 伺服器必須將每局收送的每一則訊息寫入 `--log-dir` 目錄中的一個 JSON Lines 檔案，檔名包含 `game_id`，一局一檔。每行是一筆紀錄：`ts` 為 Unix 時間（秒，可含小數），`dir` 為 `send` 或 `recv`，`peer` 為對方的 `player_id`（完成 `HELLO` 前則寫 `IP:埠號`），`msg` 為訊息物件本身；無法解析的輸入以 `{"raw": "…"}` 記錄。助教的測試工具會讀取這些檔案，因此欄位名稱與結構必須完全一致（空白與字元跳脫方式不限）。日誌目錄中只能有伺服器的日誌，每局恰好一個檔案，以 UTF-8 編碼，每筆紀錄寫入後必須立即 flush。客戶端建議以相同格式另行保存自己的日誌，位置自訂，但不得寫入伺服器的日誌目錄。範例如下：

```
{"ts": 1791474054.218, "dir": "send", "peer": "p1", "msg": {"type": "TURN",
"turn_id": 1, "guesser": "p3", "alive": ["p3", "p2", "p1"]}}
```

5.7 網際網路部署。 對玩允許跨越網際網路，因此伺服器必須位於其他組能直接連入的位址。可行的做法包括：具公開 IP 的主機、雲端虛擬機、在家用路由器設定埠轉發，或使用覆蓋式虛擬私人網路服務。校園網路可能限制外部連入，請務必提早測試。測試連通性時，macOS 與 Linux 可用 `nc -vz 主機 5040`，Windows PowerShell 可用 `Test-NetConnection 主機 -Port 5040`。伺服器暴露在網際網路上，必須具備第 16 節的防護措施。

6. 時程、繳交項目與評分方式

6.1 時程。 全學期設兩個檢查點，讓互通問題提早浮現，而不是在期末對玩當天才發現。

日期	事項	檢核內容
Oct 20, 2026	公告規範，開放參考伺服器與自我測試工具	—
Nov 10, 2026	檢查點一	本組 `client` 與 `ai` 連上參考伺服器，完成至少 3 局，繳交日誌
Dec 1, 2026	檢查點二	本組 `server` 通過自我測試工具全部項目，並與至少一組完成互通對局
Dec 15, 2026	登記對玩伺服器	繳交伺服器位址與埠號，由助教測試可否連入
Dec 22, 2026	繳交全部成果，全班跨組對玩；AI 錦標賽於前置檢查後由助教執行	第 7 節
Dec 23, 2026 至 Dec 29, 2026	展示與個人問答	依預約時段進行，每組約 20 分鐘（第 6.4 節）

6.2 繳交項目。 各組於期限前繳交下列成果：

1. 原始程式碼：Git 儲存庫連結或壓縮檔，根目錄含 `bc.py`、`requirements.txt` 與 `README.md`（安裝方式、執行命令、介面操作說明）。
2. 設計報告（PDF，建議 10–15 頁）：系統架構與並行模型、狀態機實作、錯誤處理與穩健性設計、AI 策略與至少 100 局的實驗結果、測試方法與結果、組員分工與貢獻比例，以及生成式 AI 工具使用聲明。
3. 測試證據：自我測試工具的完整輸出，以及與參考伺服器、他組對玩的日誌檔。
4. 對玩日誌：12/22 本組伺服器主持各局的日誌檔。

6.3 評分。 評分以互通與正確為重，其次才是介面與 AI 強度。助教除了使用公開的自我測試工具，也會加入未公開的測試案例，因此請依規範實作，而不是只為了通過測試。

項目	比重	評量方式
協定符合度	25%	以自我測試工具與隱藏案例自動測試伺服器；以參考伺服器測試客戶端與 AI
跨組互通	20%	12/22 本組主持與參與的對局是否順利完成、有無協定錯誤

項目	比重	評量方式
伺服器穩健性與日誌	15%	斷線、逾時、惡意輸入與多人並行下的表現；日誌完整性
人類玩家客戶端	10%	介面清楚、操作正確、輸入與網路並行
AI 代理人	15%	與參考 AI 兩人對局 50 局（輪轉順序由伺服器隨機決定）的勝率，加上跨組 AI 錦標賽的平均名次
設計報告	10%	內容完整、實驗嚴謹、論述清楚
展示與個人問答	5%	每位組員皆需能解釋本組程式

6.4 評分執行方式。 為了讓評分透明、可預期，助教依下列方式執行第 6.3 節的評分：

1. 評分環境與前置檢查：在 Ubuntu Linux 與 Python 3.10 的乾淨虛擬環境中，只執行 `pip install -r requirements.txt`。任何自動測試之前，助教先確認五項：儲存庫根目錄有 `bc.py`、`requirements.txt` 與 `README.md`；套件能在乾淨環境中安裝；`server`、`client`、`ai` 三個子命令都能啟動並接受第 5.2 節的選項；伺服器能以 `--bots 2 --players 2` 自行完成一局；伺服器在 `--log-dir` 產生日誌檔。任一項不符時，助教會通知該組在 24 小時內修正一次，修正範圍限於讓程式能啟動的最小變更，修正後總分扣 5 分；仍不符者，所有自動評分項目以 0 分計。
2. 重測與取佳：自我測試與隱藏測試最多各執行 3 次，AI 的協定檢查最多執行 2 次；每個檢查項目只要在任一次通過即算通過，某次執行中途中斷時，尚未執行的項目記為未通過。隱藏案例只檢驗本規範明確規定的行為，即以「必須」表述或直接定義為錯誤的情形；以「應」表述的建議事項不列入。
3. 客戶端與 AI 的協定檢查：除了參考伺服器，助教也以模擬伺服器檢查 `HELLO`、`COMMIT`、`GUESS`、`FEEDBACK`、`REVEAL` 的格式與時效，其間會穿插未知欄位、未知訊息類型、`CHAT`、非致命 `ERROR` 與 `PING`。
4. 人類玩家客戶端：助教依固定的操作腳本親自操作並全程錄影，評定四個面向：並行與即時性（等待輸入時仍即時顯示訊息、自動回應 `PING`、逾時後收回輸入提示）；流程正確性（自動承諾與揭示、回報預填且修改需確認、只在輪到時要求喊數、被淘汰後轉為觀看）；資訊呈現（輪轉、全部回報、淘汰、揭示判定與名次，以及剩餘時間或候選數的提示）；輸入防呆與錯誤處理（本地檢查、伺服器錯誤或斷線時顯示清楚訊息後結束程式）。介面種類不影響分數。
5. AI 對參考 AI：兩人對局 50 局，喊數、回報與承諾時限 10 秒，揭示時限 5 秒，回合上限 60；AI 須在啟動後 60 秒內完成加入。勝率 w 以第 1 名計，並列第 1 名時平分。AI 未能加入、被移出、收到 `ERROR`、棄權或被取消資格的對局稱為失誤局，一律視為落敗；失誤局超過總局數的 10% 時，此項以 0 分計，否則分數為 $\max(3, 10 \times \min(1, w \div 0.45))$，即勝率達 0.45 得滿分 10 分、能正確完成對局者至少 3 分。若 50 局的勝率低於 0.45，助教會改以 100 局重新測試，以 100 局的結果為準。

6. 跨組 AI 錦標賽：於前置檢查完成後由助教執行，採 4 至 6 人混合局，時限同上，另加入一個參考 AI，每個 AI 至少下 30 局。每局的正規化名次為 (名次 − 1) ÷ (人數 − 1)，人數指該局開局時的玩家數，取消資格或被移出記為 1；令 r 為其平均，此項分數為 $\min(5, 1 + 5 \times (1 - r))$。每一局都未能加入、或都在承諾階段就被移出的 AI 以 0 分計；前置檢查後仍無法連線的 AI 不放入錦標賽，直接以 0 分計。
7. 12/22 對玩的身分標示：連線時以 `--team` 填入本組代碼（例如 `G07`），玩家名稱以組別代碼開頭（例如 `G07-AI`、`G07-小明`）。主持組的伺服器本身算一組，其以 `--bots` 啟動的 AI 也算主持組，因此每局至少還要有另外兩組的程式連入。各組主持的日誌上傳到以組別代碼命名的目錄。
8. 跨組互通的計分：主持的一局要符合第 7 項的參與條件、以 `LAST_PLAYER` 或 `MAX_TURNS` 正常結束，而且日誌重算的所有判定都正確，才算合格。參與他組主持的對局時，本組程式每收到一次非致命 `ERROR` 就降低該局分數；收到致命錯誤、因程式問題而棄權或被取消資格，該局不計分。人類玩家因反應較慢而逾時，經助教以錄影或現場紀錄確認後，不視為程式問題；主持端誤判造成的出局，責任在主持組。所有判定一律以日誌為準。
9. 網路備援：12/15 登記時，助教會從外部測試能否連入。若 12/22 伺服器無法從網際網路連入，曾通過 12/15 測試而當天屬校園或業者網路故障者，可改在教室區網主持，不影響主持成績；從未通過連通測試者也可改在區網主持，但主持對局的分數至多一半。
10. 設計報告：除第 6.2 節的內容外，報告須附系統架構圖、伺服器狀態機及其在程式中的對應位置、AI 實驗的局數與統計誤差（標準誤或信賴區間），以及自行設計的測試與結果。報告中的說法必須與程式和日誌一致。
11. 展示與個人問答：12/23 至 12/29 依預約進行，每組約 20 分鐘。每位組員單獨回答兩題本人負責模組的問題與一題協定問題，可以開啟自己的程式碼，但其他組員不得提示。

7. 跨組對玩程序與學術誠信

7.1 對玩程序。 12/22 的對玩以「每一局都由不同組的程式組成」為原則，依下列步驟進行：

1. 各組依登記的位址開啟伺服器，`--players` 設為 4 至 6。
2. 對玩分輪進行。每輪由若干組擔任莊家，其餘各組以 `client` 或 `ai` 連入；每一局至少包含來自三個不同組的程式，莊家可另以 `--bots` 補足 AI 玩家。
3. 每組至少主持 2 局，並至少以客戶端身分參加 3 個不同組主持的對局，其中至少 1 局由人類操作。
4. 每局結束後，莊家組上傳該局日誌。對局結果或協定錯誤若有爭議，一律以日誌為準。
5. 對玩結束後，助教以各組的 AI 進行多輪 4 至 6 人混合局，以平均名次排序。

主持對局時伺服器當機，或客戶端送出不合協定的訊息，都會從日誌中被發現，並反映在「跨組互通」的成績。對方組的錯誤不會拖累本組，前提是本組程式依規範回應了那些錯誤。

7.2 學術誠信。 各組可以互相討論協定的理解，也應該彼此進行互通測試，但不得互相分享或抄襲原始程式碼。可以使用生成式 AI 工具輔助開發，但必須在報告中說明使用的工具、用途與範圍。每位組員都要能解釋所繳交的每一段程式；展示問答時無法說明的部分不予計分。

第二部分　BC/1.0 應用層通訊協定

8. 用語、傳輸層與訊息框架

8.1 規範用語。 本部分的「必須」、「不得」、「應」、「不應」與「可」，依序對應 RFC 2119 的 MUST、MUST NOT、SHOULD、SHOULD NOT 與 MAY（Bradner, 1997; Leiba, 2017）。「必須」與「不得」是互通的底線，違反就是協定錯誤；「應」允許在有充分理由時不做，但須在報告中說明。本文的「存活玩家」指尚未被猜中、棄權或取消資格的玩家；「喊數者」是本回合輪到的玩家；「回報者」是本回合除喊數者外的所有存活玩家。

8.2 傳輸層。 BC/1.0 建立在 TCP 之上（Eddy, 2022），由客戶端主動連線，預設埠號為 5040。一個連線只對應一位玩家的一局遊戲，一局結束後由伺服器關閉；客戶端要再玩一局時必須重新連線。伺服器必須支援 IPv4，可另外支援 IPv6。請特別注意：TCP 是位元組串流，不保留訊息邊界。一次 `recv()` 可能只收到半則訊息，也可能一次收到好幾則，因此需要下列的訊息框架來切分訊息。

8.3 訊息框架。 每則訊息都是一個 JSON 物件（Bray, 2017），線上格式即 JSON Lines（Ward, n.d.），細節如下：

1. 訊息以 UTF-8 編碼、不含 BOM，序列化後占用單獨一行，以換行字元 LF（0x0A）結尾。
2. 發送端不得在訊息內輸出未跳脫的換行，不得送出空行、`NaN` 或 `Infinity`，同一物件不得有重複的鍵。Python 的 `json.dumps()` 預設不會輸出未跳脫的換行，但結尾的 LF 必須自行加上，並建議設定 allow_nan=False 以免送出 NaN。
3. 接收端必須容許 LF 之前多一個 CR，並應忽略空行；含有 NaN 或 Infinity 的訊息視為無法解析。
4. 一則訊息連同結尾的 LF 不得超過 4096 位元組。
5. 每個物件必須有字串欄位 `type`，其值為全大寫的訊息類型；其餘欄位名稱一律為小寫加底線。
6. 接收端必須忽略不認識的欄位。客戶端必須忽略不認識的訊息類型；完成 HELLO 之後，伺服器收到不認識的類型時回應 `UNKNOWN_TYPE` 錯誤。
7. 伺服器必須依本文件描述的順序送出訊息；TCP 保證同一連線上的訊息依序抵達。

例如一則喊數在線上的完整位元組是 `{"type":"GUESS","turn_id":3,"guess":"0524"}` 加上一個 LF。在 Python 中，可以用 asyncio 的 `StreamReader.readline()` 或 `socket.makefile()` 切分整行（Python Software Foundation, n.d.-a, n.d.-d）。

9. 資料格式與共用欄位

下表定義各訊息共用的值。所有字串比對都區分大小寫；玩家清單依輪轉順序排列，但大廳清單依加入順序，GAME_OVER 的 ranking 依名次排列。本文所稱的「控制字元」是指 Unicode 一般類別為 C 的所有字元（Cc、Cf、Cs、Co、Cn），name 與 team 都不得包含；名稱比對不做 Unicode 正規化。player_id 每局重新從 p1 起編。

名稱	JSON 型別	格式與限制	範例
`player_id`	string	伺服器指派，`p` 加正整數，依加入順序編號	`"p3"`
`name`	string	1–20 個 Unicode 字元，不含控制字元，同局不可重複	`"阿明"`
`team`	string	1–20 個字元，建議使用組別代碼	`"G07"`
`kind`	string	`"human"` 或 `"ai"`	`"ai"`
`protocol`	string	符合正規表示式 `BC/1\.[0-9]+`	`"BC/1.0"`
`game_id`	string	伺服器產生，1–64 個英數字、底線或連字號	`"3d377330a7e0"`
`guess`、`secret`	string	恰為 4 個 ASCII 數字且兩兩相異	`"0275"`
`turn_id`	integer	自 1 起每回合加 1；JSON 布林值不是整數	`7`
`a`、`b`	integer	(a, b) 必須是 14 種合法組合之一	`1`、`2`
`commit`	string	64 個小寫十六進位字元	`"6b05c8fe…8721"`
`salt`	string	32 個小寫十六進位字元	`"2eb6594c…00df"`
各種 `*_timeout`	number	正數秒數，可為整數或小數	`60`

協定中的列舉值如下：

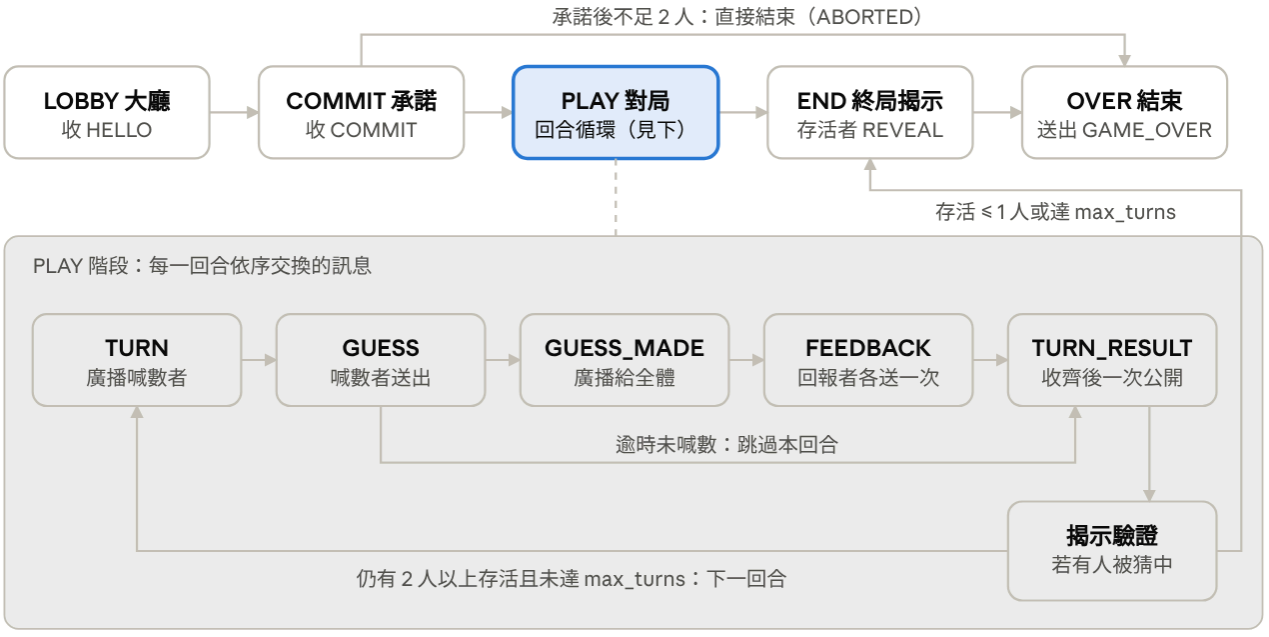
用途	合法值
終局狀態 `status`	`SURVIVOR`、`GUESSED`、`FORFEIT`、`DQ`
揭示判定 `verdict`	`OK`、`BAD_COMMIT`、`INCONSISTENT`、`NO_REVEAL`
揭示原因（`REVEAL_REQUEST.reason`）	`ELIMINATED`、`GAME_END`
結束原因（`GAME_OVER.reason`）	`LAST_PLAYER`、`MAX_TURNS`、`ABORTED`

10. 對局狀態機

伺服器一次只主持一局，依序經過五個階段。每個階段只接受特定的客戶端訊息，在錯誤階段收到的訊息，依第 13 節的檢查順序回應錯誤，通常為 `UNEXPECTED`。`CHAT`、`PING`、`PONG` 與 `BYE` 則在完成 `HELLO` 後的任何階段都可以送出。

階段	伺服器送出	接受的客戶端訊息	離開條件
`LOBBY` 大廳	`WELCOME`、`LOBBY`	`HELLO`	人數達 `--players`，或莊家輸入 `start` 且至少 2 人
`COMMIT` 承諾	`GAME_START`、`COMMITS`	`COMMIT`	全員送出承諾或逾時；剩不到 2 人時以 `ABORTED` 結束
`PLAY` 對局	`TURN`、`GUESS_MADE`、`TURN_RESULT`、`REVEAL_REQUEST`、`REVEALED`	`GUESS`、`FEEDBACK`、`REVEAL`	存活玩家不超過 1 人，或回合數達 `max_turns`
`END` 終局揭示	`REVEAL_REQUEST`、`REVEALED`	`REVEAL`	存活者完成揭示或逾時
`OVER` 結束	`GAME_OVER`	無	關閉所有連線；`--games` 未滿則回到 `LOBBY`

一局走過五個階段；對局階段由回合循環組成



對局狀態機 · 5 個階段與每回合的訊息循環

`TURN_RESULT` 之後，若有人被猜中就先完成揭示驗證，再依存活人數與回合數決定進入下一回合或終局。

11. 訊息定義

BC/1.0 共有 9 種客戶端訊息與 14 種伺服器訊息。下列兩表列出每種訊息的欄位；未標「選填」的欄位都是必填，型別與格式依第 9 節。

11.1 客戶端 → 伺服器。

訊息	允許時機	欄位	說明
`HELLO`	連線後的第一則，10 秒內送出	`protocol`、`name`、`team`、`kind`；`software`（選填，程式名稱與版本）	報名加入大廳
`COMMIT`	`COMMIT` 階段，每人一次	`commit`	秘密數字的承諾值
`GUESS`	`PLAY` 階段，輪到自己且尚未喊數	`turn_id`、`guess`	喊數
`FEEDBACK`	`PLAY` 階段，自己列在本回合 `responders` 且尚未回報	`turn_id`、`a`、`b`	回報幾 A 幾 B，一經接受即不可更改
`REVEAL`	收到 `REVEAL_REQUEST` 後	`secret`、`salt`	揭示秘密；格式錯誤時可在時限內重送
`CHAT`	`HELLO` 之後任何時候	`text`（1–200 字，不含控制字元）	聊天，由伺服器轉播
`PING`	`HELLO` 之後任何時候	`nonce`（選填）	測試連線，對方必須回 `PONG`
`PONG`	收到 `PING` 後	`nonce`（`PING` 帶有時必須原樣回傳）	回應連線測試
`BYE`	HELLO 之後任何時候	無	離開，伺服器視同斷線

11.2 伺服器 → 客戶端。

訊息	送出時機與對象	欄位
`WELCOME`	接受 `HELLO` 後，只給該玩家	`protocol`、`player_id`；`server`（選填）
`LOBBY`	大廳成員變動時，廣播	`players`（`{id, name, team, kind}` 陣列）、`min_players`（固定為 2）、`max_players`
`GAME_START`	開局時，廣播	`game_id`、`players`（依輪轉順序）、`order`、`rules`（`guess_timeout`、`feedback_timeout`、`commit_timeout`、`reveal_timeout`、`max_turns`、`max_skips`）
`COMMITS`	承諾階段結束，廣播	`order`（實際參賽者）、`commits`（`player_id` → 承諾值的物件）
`TURN`	每回合開始，廣播	`turn_id`、`guesser`、`alive`
`GUESS_MADE`	接受喊數後，廣播	`turn_id`、`guesser`、`guess`、`responders`
`TURN_RESULT`	回合結束，廣播	`turn_id`、`guesser`、`guess`（跳過時為 `null`）、`skipped`（布林）、`results`（`{player, a, b}` 陣列）、`eliminated`、`forfeited`、`disqualified`
`REVEAL_REQUEST`	只給需要揭示的玩家	`reason`
`REVEALED`	每完成一筆揭示驗證，廣播	`player`、`secret`、`salt`（未揭示時皆為 `null`）、`verdict`、`bad_turns`（整數陣列）
`GAME_OVER`	一局結束，廣播	`game_id`、`reason`、`turns`、`ranking`（`{player, status, out_turn, rank}` 陣列）
`CHAT`	轉播聊天，廣播	`from`、`text`
`PING`、`PONG`	同客戶端	`nonce`（選填）
`ERROR`	只給出錯的連線	`code`、`message`（人類可讀的說明）、`fatal`（布林）

11.3 欄位語意。 「廣播」的對象是所有已完成 `HELLO` 且仍連線的玩家，包括已被猜中、棄權或取消資格但仍在線上者，只有在承諾階段被移出的玩家不在其內。`TURN.alive` 包含喊數者本人；`GUESS_MADE.responders` 是 `alive` 去掉喊數者。`TURN_RESULT.results` 只列出在時限內回報者，三個名單欄位與 `results` 都依輪轉順序排列。`GAME_OVER.ranking` 依名次排列，取消資格者置於最後且 `rank` 為 `null`；存活者的 `out_turn` 為 `null`，其餘為離場的回合編號。`ERROR` 的範例如下：

```
{"type":"ERROR","code":"NOT_YOUR_TURN","message":"本回合喊數者為
p2","fatal":false}
```

名次相同的玩家與取消資格的玩家，在 `ranking` 中各自依輪轉順序排列。取消資格者的 `out_turn` 為被判定的回合，終局揭示失敗者則為最後一個回合編號。在承諾階段被移出的玩家不出現在 `ranking` 中。`REVEALED.bad_turns` 只在判定為 `INCONSISTENT` 時非空，其餘判定一律為空陣列。`PING` 與 `PONG` 的 `nonce` 為字串。`LOBBY.max_players` 即伺服器的 `--players`。

12. 伺服器處理程序

伺服器必須依下列程序主持一局。除了 HELLO 的 10 秒時限從接受連線時起算，其餘時限皆從伺服器送出對應訊息的時刻起算。由於跨網際網路有傳輸延遲，客戶端應至少提前 1 秒送出，而不要卡在時限邊緣。

12.1 連線與大廳。

1. 接受連線後開始計時，10 秒內未收到合法的 `HELLO` 則回應 `TIMEOUT` 並斷線。
2. 完成 `HELLO` 之前，無法解析的行回應非致命的 `BAD_JSON`（計時照算）；其他任何訊息，包括 `PING`、`BYE` 與不認識的類型，一律回應 `UNEXPECTED` 並斷線。
3. `HELLO` 依序檢查：欄位是否齊全（`BAD_FIELD`）、版本（`protocol` 不符合 `BC/1\.[0-9]+` 者一律為 `VERSION_UNSUPPORTED`）、值（`INVALID_VALUE`）、是否在大廳階段（`GAME_IN_PROGRESS`）、人數（`ROOM_FULL`）、名稱（`NAME_TAKEN`）；這些錯誤全部是致命錯誤；已完成 HELLO 的連線再次送出 HELLO 時，則回應非致命的 UNEXPECTED，連線保留。
4. 通過檢查後指派 `player_id`（每局從 `p1` 起編），回應 `WELCOME`，再廣播 `LOBBY`。
5. 大廳中斷線的玩家直接移除，並廣播新的 `LOBBY`。
6. 人數達到 `--players`，或莊家輸入 `start` 且至少 2 人時，進入承諾階段。

12.2 承諾階段。

1. 隨機排列輪轉順序，廣播 `GAME_START`。
2. 等待每位玩家的 `COMMIT`，至多 `commit_timeout` 秒。
3. 時限到或全員送出後，將未送出承諾或已斷線者移出本局：仍連線者收到致命的 `TIMEOUT` 錯誤後斷線，被移出者不列名次。
4. 剩下不足 2 人時，廣播 `GAME_OVER`（`reason` 為 `ABORTED`、`turns` 為 0、`ranking` 為空陣列）。否則廣播 `COMMITS`，進入對局。

12.3 每一回合。

1. `turn_id` 加 1。喊數者是輪轉順序中「上一位喊數者之後」的第一位存活玩家，第一回合為 COMMITS 訊息中 `order` 的第一位。廣播 `TURN`。
2. 等待喊數者的 `GUESS`，至多 `guess_timeout` 秒；喊數者斷線則立即停止等待。

3. 未收到喊數時，本回合跳過。喊數者若仍連線，其連續跳過次數加 1，達 `max_skips` 即棄權。
4. 收到喊數時，喊數者的連續跳過次數歸零；以 `alive` 去掉喊數者作為回報者，廣播 `GUESS_MADE`。
5. 等待所有回報者的 `FEEDBACK`，至多 `feedback_timeout` 秒；已斷線者不再等待。收齊之前不得透露任何回報。
6. 依輪轉順序處理每位回報者。未回報者判為棄權。有回報者把 (`turn_id`, `guess`, a, b) 加入其公開紀錄並重算候選集合：候選集合為空者取消資格；否則若回報為 4A0B，判為被猜中。
7. 此時仍存活但已斷線的玩家，包括喊數後才斷線的喊數者，一併判為棄權。
8. 將本回合離場者的 `out_turn` 設為 `turn_id`，廣播 `TURN_RESULT`。
9. 本回合若有人被猜中，對其中仍連線者送出 `REVEAL_REQUEST`（`ELIMINATED`），等待 `REVEAL` 至多 `reveal_timeout` 秒，再依輪轉順序逐一驗證並廣播 `REVEALED`；驗證失敗者改判為取消資格。
10. 存活者不超過 1 人時以 `LAST_PLAYER` 進入終局；`turn_id` 等於 `max_turns` 時以 `MAX_TURNS` 進入終局；否則回到步驟 1。

12.4 終局。

1. 對所有存活玩家送出 `REVEAL_REQUEST`（`GAME_END`），等待、驗證並逐一廣播 `REVEALED`；驗證失敗者改判為取消資格。
2. 依第 14 節計算名次，廣播 `GAME_OVER`。
3. 送完訊息後關閉本局所有連線。

12.5 客戶端的對應義務。 客戶端收到以自己為 `guesser` 的 `TURN` 時送出一次 `GUESS`；收到自己列在 `responders` 的 `GUESS_MADE` 時送出一次 `FEEDBACK`；收到 `REVEAL_REQUEST` 時立即自動送出 `REVEAL`；收到 `PING` 時回 `PONG`。客戶端不得假設訊息間的時間間隔，也必須容許 `CHAT`、`LOBBY`、`PING` 穿插在任何兩則訊息之間。

12.6 斷線的判定。 存活玩家在對局階段斷線時，伺服器不另外送出訊息，而是在下一個判定點處理。斷線發生在某回合的 `TURN_RESULT` 送出之前，該玩家列入該回合的 `forfeited`。斷線發生在 `TURN_RESULT` 之後（例如揭示等待期間），下一回合照常開始且仍把他列在 `alive` 中；輪到他喊數時視同立即逾時，並在該回合的 `TURN_RESULT` 中判為棄權。已被要求揭示的玩家（被猜中者與終局存活者）若因斷線而未揭示，判定為 `NO_REVEAL` 並取消資格。`BYE` 的效果與斷線相同；`GAME_OVER` 之後送達的訊息一律忽略。

13. 錯誤碼與檢查順序

伺服器收到不合協定的訊息時，只回應一則 `ERROR` 給該連線，並忽略該訊息，遊戲狀態不變。非致命錯誤發生後，客戶端可以在時限內修正並重送；`fatal` 為 `true` 時，伺服器送出錯誤後立即關閉連線。客戶端不得對伺服器回應 `ERROR`。

錯誤碼	致命	情境
`BAD_JSON`	否	無法解析為 JSON、不是物件，或缺少字串型別的 `type`
`LINE_TOO_LONG`	是	單行超過 4096 位元組
`UNKNOWN_TYPE`	否	不認識的訊息類型
`BAD_FIELD`	在 `HELLO` 時為是	必填欄位缺漏或 JSON 型別錯誤，包括以布林值充當整數
`INVALID_VALUE`	在 `HELLO` 時為是	型別正確但值不合法，例如數字重複、回報 3A1B、承諾值格式錯誤
`UNEXPECTED`	在 `HELLO` 之前為是	目前的階段或身分不允許此訊息
`NOT_YOUR_TURN`	否	非喊數者送出 `GUESS`
`STALE_TURN`	否	`turn_id` 不是目前回合
`DUPLICATE`	否	重複送出 `COMMIT`、`FEEDBACK` 或 `REVEAL`
`VERSION_UNSUPPORTED`	是	協定主版本不是 1
`GAME_IN_PROGRESS`	是	在大廳以外的階段要求加入
`ROOM_FULL`	是	人數已滿
`NAME_TAKEN`	是	名稱與同局玩家重複
`TIMEOUT`	是	10 秒內未送 `HELLO`，或承諾逾時而被移出本局
`TOO_MANY_ERRORS`	是	同一連線累積超過 20 次錯誤

同一則訊息可能同時觸犯多個錯誤，伺服器必須依下列順序檢查，只回應第一個符合的錯誤碼，讓測試結果具有唯一性：

1. `COMMIT`：`BAD_FIELD` → `UNEXPECTED`（不在承諾階段，或非本局參賽者）→ `DUPLICATE` → `INVALID_VALUE`。
2. `GUESS`：`BAD_FIELD` → `UNEXPECTED`（不在對局階段）→ `STALE_TURN` → `NOT_YOUR_TURN` → `UNEXPECTED`（本回合已喊數或已逾時）→ `INVALID_VALUE`。
3. `FEEDBACK`：`BAD_FIELD` → `UNEXPECTED`（不在對局階段）→ `STALE_TURN` → `UNEXPECTED`（不是本回合回報者，或回報已截止）→ `DUPLICATE` → `INVALID_VALUE`。
4. `REVEAL`：`BAD_FIELD` → `DUPLICATE`（已揭示過）或 `UNEXPECTED`（不需要揭示）→ `INVALID_VALUE`。
5. `CHAT`：`BAD_FIELD` → `INVALID_VALUE`。

格式正確但內容不實的 `REVEAL`（例如揭示的秘密與承諾值不符）不是協定錯誤，而是由 `REVEALED.verdict` 判定為違規。

同一連線的第 21 次錯誤發生時，伺服器先送出該次的錯誤，再送出致命的 `TOO_MANY_ERRORS` 並斷線；這是「一則訊息只回應一則 `ERROR`」的唯一例外。

14. 揭示驗證與名次計算演算法

14.1 公開一致性檢查。 伺服器為每位玩家 X 維護候選集合 C(X)，初始為全部 5040 個合法數字。每收到 X 的一筆回報 (g, a, b)，就保留與之相容的數字；若 C(X) 變為空集，X 於本回合取消資格。AI 代理人推論對手秘密時用的是同一個運算：

```
cands[x] = [s for s in cands[x] if score(guess, s) == (a, b)]
if not cands[x]:
    disqualify(x)          # 不存在任何秘密數字能解釋 X 的全部回報
```

14.2 揭示驗證。 對玩家 X，設其承諾值為 c(X)、公開回報紀錄為 R(X)（每筆為回合編號 t、喊數 g 與回報 a、b）、揭示內容為 (s, r)。伺服器與客戶端依下列步驟得出判定：

1. 未在時限內收到格式正確的 `REVEAL`：判定 `NO_REVEAL`。
2. SHA256(s + ":" + r) 不等於 c(X)：判定 `BAD_COMMIT`。
3. 對 R(X) 的每一筆紀錄重算 score(g, s)，與回報的 (a, b) 不同者將 t 加入 `bad_turns`；`bad_turns` 非空則判定 `INCONSISTENT`。
4. 其餘情況判定 `OK`。

步驟 3 同時涵蓋兩種作弊：被淘汰者的秘密必須等於猜中他的那個喊數，因為只有 g = s 時才會得到 4A0B；存活者則不可能曾經隱瞞過 4A。

14.3 名次計算。 依第 2.4 節的規則，名次計算步驟如下：

1. 取消資格（`DQ`）者先行排除。
2. 其餘每位玩家給一個排序鍵：`SURVIVOR` 為 (0, 0)，`GUESSED` 為 (1, −out_turn)，`FORFEIT` 為 (2, −out_turn)。
3. 依鍵遞增排序；每位玩家的名次等於 1 加上「鍵嚴格小於自己的玩家數」，因此鍵相同者並列。
4. 取消資格者附在最後，`rank` 為 `null`。

```
def compute_ranking(entries):            # entries: [(player_id, status,
out_turn)]
    cat = {"SURVIVOR": 0, "GUESSED": 1, "FORFEIT": 2}
    key = lambda e: (cat[e[1]], 0 if e[1] == "SURVIVOR" else -e[2])
    ranked = sorted((e for e in entries if e[1] != "DQ"), key=key)
    keys = [key(e) for e in ranked]
    rows = [(e[0], e[1], e[2], 1 + sum(k < key(e) for k in keys)) for e in
ranked]
    return rows + [(e[0], e[1], e[2], None) for e in entries if e[1] == "DQ"]
```

15. 完整對局範例

下列是參考伺服器上一局實際對局的完整訊息紀錄，由該局的伺服器日誌擷取而成。三位玩家都是參考 AI，分別以 G03、G07、G12 的名義連線，伺服器的喊數與回報時限設為 30 秒；這裡只列出 p1（G03-AI）這條連線上的訊息。`C→S` 是 p1 送出的訊息，`S→C` 是伺服器送給 p1 的訊息；線上實際傳送時沒有這個前綴，只有 JSON 與結尾的 LF。

```
C→S {"type":"HELLO","protocol":"BC/1.0","name":"G03-
AI","team":"G03","kind":"ai","software":"BC-ref/0.9"}
S→C {"type":"WELCOME","protocol":"BC/1.0","player_id":"p1","server":"BC-
ref/0.9"}
S→C {"type":"LOBBY","players":[{"id":"p1","name":"G03-
AI","team":"G03","kind":"ai"}],"min_players":2,"max_players":3}
S→C {"type":"LOBBY","players":[{"id":"p1","name":"G03-
AI","team":"G03","kind":"ai"},{"id":"p2","name":"G07-
AI","team":"G07","kind":"ai"}],"min_players":2,"max_players":3}
S→C {"type":"LOBBY","players":[{"id":"p1","name":"G03-
AI","team":"G03","kind":"ai"},{"id":"p2","name":"G07-
AI","team":"G07","kind":"ai"},{"id":"p3","name":"G12-
AI","team":"G12","kind":"ai"}],"min_players":2,"max_players":3}
S→C {"type":"GAME_START","game_id":"3d377330a7e0","players":
[{"id":"p3","name":"G12-AI","team":"G12","kind":"ai"},{"id":"p2","name":"G07-
AI","team":"G07","kind":"ai"},{"id":"p1","name":"G03-
AI","team":"G03","kind":"ai"}],"order":["p3","p2","p1"],"rules":
{"guess_timeout":30,"feedback_timeout":30,"commit_timeout":30,"reveal_timeout
":15,"max_turns":120,"max_skips":3}}
C→S
{"type":"COMMIT","commit":"6b05c8fe4113aa24833fcbd089d33f0fe48ff68eb0a7d7fff8
7319a394338721"}
S→C {"type":"COMMITS","order":["p3","p2","p1"],"commits":
{"p3":"60e3dc3a105e21cbefd6fe0a98afa4dda527504ff4e627f1fecb471f62d87f86","p2"
:"b3d63d828ba751989c49aeb8540a6b9d0a6c3af67a31f77f4cf4ce3210cc6f3a","p1":"6b0
5c8fe4113aa24833fcbd089d33f0fe48ff68eb0a7d7fff87319a394338721"}}
S→C {"type":"TURN","turn_id":1,"guesser":"p3","alive":["p3","p2","p1"]}
S→C
```

```
{"type":"GUESS_MADE","turn_id":1,"guesser":"p3","guess":"0927","responders":
["p2","p1"]}
C→S {"type":"FEEDBACK","turn_id":1,"a":1,"b":2}
S→C
{"type":"TURN_RESULT","turn_id":1,"guesser":"p3","guess":"0927","skipped":fal
se,"results":[{"player":"p2","a":2,"b":0},
{"player":"p1","a":1,"b":2}],"eliminated":[],"forfeited":[],"disqualified":
[]}
S→C {"type":"TURN","turn_id":2,"guesser":"p2","alive":["p3","p2","p1"]}
S→C
{"type":"GUESS_MADE","turn_id":2,"guesser":"p2","guess":"0273","responders":
["p3","p1"]}
C→S {"type":"FEEDBACK","turn_id":2,"a":3,"b":0}
S→C
{"type":"TURN_RESULT","turn_id":2,"guesser":"p2","guess":"0273","skipped":fal
se,"results":[{"player":"p3","a":1,"b":0},
{"player":"p1","a":3,"b":0}],"eliminated":[],"forfeited":[],"disqualified":
[]}
S→C {"type":"TURN","turn_id":3,"guesser":"p1","alive":["p3","p2","p1"]}
C→S {"type":"GUESS","turn_id":3,"guess":"0524"}
S→C
{"type":"GUESS_MADE","turn_id":3,"guesser":"p1","guess":"0524","responders":
["p3","p2"]}
S→C
{"type":"TURN_RESULT","turn_id":3,"guesser":"p1","guess":"0524","skipped":fal
se,"results":[{"player":"p3","a":1,"b":2},
{"player":"p2","a":2,"b":0}],"eliminated":[],"forfeited":[],"disqualified":
[]}
S→C {"type":"TURN","turn_id":4,"guesser":"p3","alive":["p3","p2","p1"]}
S→C
{"type":"GUESS_MADE","turn_id":4,"guesser":"p3","guess":"0271","responders":
["p2","p1"]}
C→S {"type":"FEEDBACK","turn_id":4,"a":3,"b":0}
S→C
{"type":"TURN_RESULT","turn_id":4,"guesser":"p3","guess":"0271","skipped":fal
se,"results":[{"player":"p2","a":0,"b":2},
{"player":"p1","a":3,"b":0}],"eliminated":[],"forfeited":[],"disqualified":
[]}
S→C {"type":"TURN","turn_id":5,"guesser":"p2","alive":["p3","p2","p1"]}
S→C
{"type":"GUESS_MADE","turn_id":5,"guesser":"p2","guess":"0274","responders":
["p3","p1"]}
C→S {"type":"FEEDBACK","turn_id":5,"a":3,"b":0}
S→C
{"type":"TURN_RESULT","turn_id":5,"guesser":"p2","guess":"0274","skipped":fal
se,"results":[{"player":"p3","a":1,"b":1},
```

```
{"player":"p1","a":3,"b":0}],"eliminated":[],"forfeited":[],"disqualified":
[]}
S→C {"type":"TURN","turn_id":6,"guesser":"p1","alive":["p3","p2","p1"]}
C→S {"type":"GUESS","turn_id":6,"guess":"3527"}
S→C
{"type":"GUESS_MADE","turn_id":6,"guesser":"p1","guess":"3527","responders":
["p3","p2"]}
S→C
{"type":"TURN_RESULT","turn_id":6,"guesser":"p1","guess":"3527","skipped":fal
se,"results":[{"player":"p3","a":0,"b":1},
{"player":"p2","a":4,"b":0}],"eliminated":["p2"],"forfeited":
[],"disqualified":[]}
S→C
{"type":"REVEALED","player":"p2","secret":"3527","salt":"8cccf65853321415f264
51c5a5d016f4","verdict":"OK","bad_turns":[]}
S→C {"type":"TURN","turn_id":7,"guesser":"p3","alive":["p3","p1"]}
S→C
{"type":"GUESS_MADE","turn_id":7,"guesser":"p3","guess":"0275","responders":
["p1"]}
C→S {"type":"FEEDBACK","turn_id":7,"a":4,"b":0}
S→C
{"type":"TURN_RESULT","turn_id":7,"guesser":"p3","guess":"0275","skipped":fal
se,"results":[{"player":"p1","a":4,"b":0}],"eliminated":["p1"],"forfeited":
[],"disqualified":[]}
S→C {"type":"REVEAL_REQUEST","reason":"ELIMINATED"}
C→S
{"type":"REVEAL","secret":"0275","salt":"2eb6594c74b9b9d71e2d9b5bd14300df"}
S→C
{"type":"REVEALED","player":"p1","secret":"0275","salt":"2eb6594c74b9b9d71e2d
9b5bd14300df","verdict":"OK","bad_turns":[]}
S→C
{"type":"REVEALED","player":"p3","secret":"0415","salt":"22fb8a673d8f4ac49e3f
5748d9c5a94b","verdict":"OK","bad_turns":[]}
S→C
{"type":"GAME_OVER","game_id":"3d377330a7e0","reason":"LAST_PLAYER","turns":7
,"ranking":[{"player":"p3","status":"SURVIVOR","out_turn":null,"rank":1},
{"player":"p1","status":"GUESSED","out_turn":7,"rank":2},
{"player":"p2","status":"GUESSED","out_turn":6,"rank":3}]}
```

這局驗證了協定的幾個關鍵細節。第 6 回合 p1 喊出 `3527`，恰為 p2 的秘密，p2 回報 4A0B 而淘汰。伺服器只對 p2 送出 `REVEAL_REQUEST`，所以 p1 的紀錄中看不到那則訊息，只看得到廣播的 `REVEALED`。第 7 回合輪到 p3 喊數，而不是剛剛猜中人的 p1，這就是「無論猜中與否都輪到下一位」的規則。p1 被猜中後自動揭示，其秘密與鹽值正是第 3.1 節的測試向量。最後存活的 p3 在終局揭示並通過驗證，獲得第 1 名；p1 比 p2 晚一回合離場，故名列第 2。

16. 安全性、穩健性、版本相容與常見實作陷阱

16.1 安全性與穩健性。 伺服器會暴露在網際網路上，必須假設任何連入者都可能送出錯誤或惡意的資料，因此必須做到以下各項：

1. 只用 `json.loads()` 解析網路資料，絕不可用 `eval`、`exec` 或 `pickle`。
2. 限制單行長度，也限制每個連線待送出的訊息量。不讀取資料的客戶端不得阻塞對其他人的廣播，建議每個連線配置獨立的傳送佇列。
3. 每個連線的例外都要攔截；單一連線出錯只關閉該連線，不得影響對局。
4. 所有計時由伺服器掌握，不採信客戶端提供的任何時間資訊。
5. 同一連線累積超過 20 次錯誤即斷線。

BC/1.0 以明文傳輸，傳輸層加密（TLS）不在本版範圍內。秘密數字在揭示前從不上線，揭示時遊戲也已不再需要保密，所以明文傳輸不影響公平性；但請勿在聊天中傳送個人資料。

16.2 版本相容。 協定版本字串為 `BC/主版本.次版本`。未來的次版本只會新增選填欄位或新訊息類型，不改變既有語意，因此伺服器必須接受任何 `BC/1.x`，接收端必須忽略不認識的欄位。客戶端應檢查 `WELCOME.protocol` 的主版本是否為 1。

16.3 常見實作陷阱。 下列問題容易造成互通失敗，實作時請逐項檢查：

1. 把一次 `recv()` 當成一則訊息。應累積緩衝區，依 LF 切出完整的行，並保留剩餘的位元組。
2. 在主迴圈呼叫會阻塞的 `input()`，導致等待輸入時無法收訊、無法回 `PONG`、錯過 `TURN`。應以執行緒或 asyncio 分開輸入與網路；圖形介面則在事件迴圈中處理網路事件。
3. 在 Python 中 `isinstance(True, int)` 為 `True`，檢查整數欄位時必須排除布林值。
4. `str.isdigit()` 會接受全形數字「４」等非 ASCII 數字，應改用 re.fullmatch() 搭配正規表示式 `[0-9]{4}` 檢查。
5. 使用 `socket.send()` 而未處理部分送出的情況，應改用 `sendall()`。
6. 在繁體中文版 Windows 上，open() 與 socket.makefile() 未指定 encoding 時會採用系統地區編碼（通常是 cp950），而不是 UTF-8（Python Software Foundation, n.d.-b），中文名稱與日誌因此出錯。網路資料與日誌檔一律明確指定 encoding="utf-8"；主控台若仍出現亂碼，可用 `sys.stdout.reconfigure(encoding="utf-8")`。
7. 以 `random` 產生鹽值。`random` 的輸出可被預測，必須改用 `secrets`。
8. AI 只用自己喊數的結果更新候選，浪費了其他玩家創造的資訊。
9. 以為 `TURN_RESULT` 之後一定緊接著下一個 `TURN`。中間可能出現 `REVEAL_REQUEST`、`REVEALED` 或 `CHAT`。
10. 伺服器在回報收齊前就轉發個別回報，違反第 3 節的公平原則。

17. 課程提供的參考伺服器與自我測試工具

課程提供兩項資源，讓各組不必等到別組完成，就能驗證自己的互通性。第一項是公開運行的參考伺服器（位址另行公告），可加入參考 AI 玩家，用來測試各組的 `client` 與 `ai`。參考伺服器的原始碼不公開。

第二項是協定自我測試工具 `bc_selftest.py`，原始碼公開。它以腳本化的 TCP 客戶端對受測伺服器送出正常與異常的訊息，完整執行共 48 項檢查，範圍包括：2 至 6 人的純 AI 對局各 10 局、連線與欄位錯誤、說謊、揭示不符、斷線、回報逾時、連續未喊數與承諾逾時。參考實作通過全部 48 項。檢查項目的名稱與數量固定，某一節中途出錯時，該節尚未執行的項目記為失敗，工具本身不會中斷；工具每次啟動受測伺服器都改用一個尚未被占用的埠號，自 5600 起約使用 15 個。用法如下，加上 `--skip-bots` 可略過 50 局純 AI 對局，只執行其餘 38 項；加上 `--json 檔名` 可把逐項結果存成 JSON 檔，作為報告的測試證據：

```
python bc_selftest.py --server "python 你的專案/bc.py server"
```

自我測試工具會以第 5.2 節的命令列選項啟動受測伺服器，並讀取第 5.6 節格式的日誌，所以這兩節的要求也在測試範圍內。下表是參考實作在本機進行 50 局純 AI 對局的回合數，可用來估計一局的長度；每種人數僅 10 局，不宜作為 AI 強弱的比較基準。

玩家數	局數	最少回合	平均回合	最多回合
2	10	5	8.1	10
3	10	7	9.4	11
4	10	7	9.4	12
5	10	9	10.6	12
6	10	10	12.3	15

References

Blum, M. (1983). Coin flipping by telephone a protocol for solving impossible problems. *ACM SIGACT News, 15*(1), 23–27. https://doi.org/10.1145/1008908.1008911

Bradner, S. (1997). *Key words for use in RFCs to indicate requirement levels* (RFC 2119). Internet Engineering Task Force. https://doi.org/10.17487/RFC2119

Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) data interchange format* (RFC 8259). Internet Engineering Task Force. https://doi.org/10.17487/RFC8259

Eddy, W. (Ed.). (2022). *Transmission Control Protocol (TCP)* (RFC 9293). Internet Engineering Task Force. https://doi.org/10.17487/RFC9293

Knuth, D. E. (1976). The computer as master mind. *Journal of Recreational Mathematics, 9*(1), 1–6.

Kooi, B. (2005). Yet another mastermind strategy. *ICGA Journal, 28*(1), 13–20. https://doi.org/10.3233/ICG-2005-28105

Leiba, B. (2017). *Ambiguity of uppercase vs lowercase in RFC 2119 key words* (RFC 8174). Internet Engineering Task Force. https://doi.org/10.17487/RFC8174

National Institute of Standards and Technology. (2015). *Secure hash standard (SHS)* (FIPS PUB 180-4). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.180-4

Python Software Foundation. (n.d.-a). *asyncio — Streams*. Python 3 documentation. Retrieved October 8, 2026, from https://docs.python.org/3/library/asyncio-stream.html

Python Software Foundation. (n.d.-b). *Built-in functions*. Python 3 documentation. Retrieved October 9, 2026, from https://docs.python.org/3/library/functions.html

Python Software Foundation. (n.d.-c). *secrets — Generate secure random numbers for managing secrets*. Python 3 documentation. Retrieved October 8, 2026, from https://docs.python.org/3/library/secrets.html

Python Software Foundation. (n.d.-d). *socket — Low-level networking interface*. Python 3 documentation. Retrieved October 8, 2026, from https://docs.python.org/3/library/socket.html

Tanaka, T. (n.d.). *The minimum strategy and the strongest strategy of the number guessing game MOO*. University of Tokyo. Retrieved October 8, 2026, from https://www.tanaka.ecc.u-tokyo.ac.jp/ktanaka/moo/moo-en.html

Ward, I. (n.d.). *JSON Lines*. Retrieved October 8, 2026, from https://jsonlines.org/

Wikipedia contributors. (n.d.). Bulls and cows. In *Wikipedia*. Retrieved October 8, 2026, from https://en.wikipedia.org/wiki/Bulls_and_cows